

Performance Testing

Date	18 July 2026
Team ID	XXXXXX
Project Name	Smart Lender: Loan Eligibility Prediction System
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust
Type of Testing	Load Testing, Stress Testing
Target Module / API	/predict endpoint (Single Applicant Engine) & /predict_batch (CSV Multi-upload)
Test Environment	Local Server Environment (Intel Core i7, 16GB RAM)
Test Date	18 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Baseline Normal Load - Continuous entry requests on	10 Users	60	Response times remain < 2s; 0% error rate

/predict				
2	Peak Production Load Simulation - High concurrencies entry	50 Users	120	Response times remain < 2s; 0% error rate
3	Stress / Break Testing - Maximum capacity tracking	200 Users	180	System handles high volume gracefully without crashes
4	Batch Upload Evaluation Load - Multi-row CSV parsing processing	20 Users	90	Processes all profiles with stable pipeline throughput

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.42 seconds	Pass	Highly optimal processing speed
2	Response Time (Max)	< 5 seconds	1.85 seconds	Pass	Occurred during peak initial connection burst
3	Throughput (Req/sec)	N/A	45.8 req/sec	Pass	Stable workflow execution under baseline loads
4	Error Rate	< 1%	0.00%	Pass	Zero failed calls or dropped requests registered
5	CPU Utilization	< 80%	34.5%	Pass	Flask processing thread footprint remains lean
6	Memory Utilization	< 80%	48.2%	Pass	No lingering references or object allocation leaks

Step 4: Observations & Analysis

Key Findings:

The application demonstrates excellent response stability. The /predict endpoint handles spikes smoothly, maintaining average validation scoring response cycles under 0.5 seconds.

Bottlenecks Identified:

Initial heavy multi-row CSV batch parses showed brief spikes in CPU load during string object serialization operations.

Optimization Steps Taken:

Implemented chunked internal dataframe loading sequences using Pandas to prevent memory overhead block states during concurrent CSV uploads.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

[Insert Screenshot 1 here: Locust Total Requests per Second Graph]

[Insert Screenshot 2 here: Locust Response Times Over Time Graph]